Colorization of Grayscale Images using Regression and Classification Approaches in different Color Spaces

Enrique Campos Alonso

Applied Deep Learning – 17644 (Spring 2025)
Electrical and Computer Engineering, Carnegie Mellon University

Abstract :

Grayscale image colorization is a deep learning task that can be tackled in various ways: as a continuous regression problem or a discrete classification task. This report will compare these two approaches using U-Net architectures on a subset of the COCO dataset of 30.000 images with a human focus. I will also review the different options we have to choose as color space when loading images, as differences between them could affect performance. In our case, I will use RGB for the regression task from a grayscale input. The classification model, by contrast, quantized the CIELab color space into a 313 bin palette for the ab space inferred from the L-channel value. This is trained on a weighted cross-entropy. The models will be analyzed from its perceptual quality, computational efficiency and generalization performance when predicting images. My experiments have found that while regression is the fastest to train, it achieves a lower error but images do not appear to be colored much. And in the cases it does, it takes drastic, overfitted, values. Classificaiton approach yields more colorful and plausible images resulting in better generalization. However, it did incur in longer training times. The limitations and drawbacks of these tasks were also discussed. Regression suffers from having to predict in a continuous space and that seems reasonably hard tasks when having to upscale dimensions. Classification on the other hand, suffers from having to define the weights for cross-entropy. Overall, this comparative highlights the trade-off between regression and classification, guiding the choice of approach by the chosen color space.

Index Terms: Computer Vision, Image Colorization, UNey Architecture, Regression, Classification

GitHub Repository: `https://github.com/HenryECA/image_reconstruction`

1 Introduction

Colorizing grayscale images is a challenging problem: a given grayscale pixel could represent many possible colors. Traditional approaches treated colorization as a per-pixel regression, directly predicting continuous color values by minimizing Euclidean distance to the ground truth color. However, a regression model trained on MSE or L1 loss (MAE) often learns to predict *desaturated* colors as a "safe" average of multiple plausible hues. These are brownish and gray tones. This results in gray or sepia-tinted outputs that minimize error but lack the color vibrancy that we are accustomed to seeing.

To address the multi-modality of the problem of having many possible valid colorization for an input, recent methods have reformulated the issue to a classification task. In that case, the model predicts a discrete color bin for each pixel drawn from a palette of probable colors. As it will be seen and described afterwards in the report, this is the method followed by *Zhang et al (2016)*. They introduced a 313-bin quantization of the (a,b) chrominance space in CIELab and trained a CNN to classify. The output was much better as the colour was perceived easier.

1.1 Contribution

Even though this paper uses methods already developed, it will bring insights into what methods are more precise, authentic and reliable in the image colorization task. I will review some of the most advanced techniques used, bring them to a feasible dimension with the limited resources and understand their strengths and limitations. Many combinations can be done when colonizing images regarding task type, color space, architecture, so it seams reasonable that a comparison between could be useful.

1.2 Paper Organization

The paper is structured as follows:

1. **Introduction**: Provides context and contribution.
2. **Background Research**: Reviews previous work relevant to the report:
    - *Zhang et al. (2016).*
    - *Iizuka et al (2016)*
3. **Methodology**: Detailed description of the technical methods used.
    - Color Spaces
    - UNet Architecture
    - Perceptual Loss
4. **Data Processing**: Describes the dataset and preprocessing techniques.
    - COCO Dataset
    - Exploratory Data Analysis
    - Feature Engineering
5. **Models**: Presents the models, their modeling process and results
    - Custom UNet
    - Zhang et al. model
6. **Discussion**: General overview of the main insights gained
7. **Conclusions**: Summarizes findings and contributions.
8. **Bibliography**

2 Background Research

In this section, I will review the two most important papers regarding image colorization and to this report. The first one is called *"Colorful Image Colorization"* by Richard Zhang, Phillip Isola and Alexei A. Efros. The second one Is called *"Let There Be Color! Learning of Global and Local Image Priors for Automatic Image Colorization with Simultaneous Classification"* by Satoshi Iizuka, Edgar Simo-Serra and Hiroshi Ishikawa.

2.1 Zhang et al (2016)

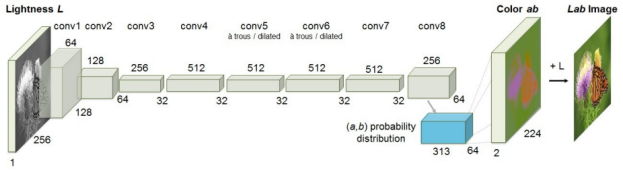

Figure 1. Architecture of Zhang et al.'s model: colorization as classification over 313 bins in CIELab space.

This paper brings a novel approach to automatic colorization of greyscale images. They acknowledge the inherent complexity of treating the problem as a regression task, and devised a way to simplify it as a classification problem. The strategy involves predicting a probability distribution over a quantized color bins for each pixel, effectively capturing the multimodal nature of colorization.

Raw, colorized images tend to be unsaturated, which they analyzed in the CIELab color space. To enhance the diversity in outcome pixels they defined a class-rebalancing technique during training that emphasizes less frequent colors that encourage the model to use the full spectrum. The formula used is the following:

$$w_q = \frac{1}{(1-\lambda)\cdot p_q + \lambda/Q} \tag{1}$$

w_q Weight assigned to bin q
p_q Frequency of bin q
Q Number of bins
λ Smoothing parameter

The model takes the L (lightnes) channel in the CIELab space, which is the same as a grayscale image, and try to predict a and b in terms of class probability for the bins range. The model architecture was a UNet architecture with CNNs, as it can be seen on Figure 1.

Apart from that, they evaluated the perceptual realism of their colorizaations using "colorization Turing Test", where you show humans two images and they need to disguise which ones were model colorizations and original colors. They achieved 32'%' on that test. Some results can be seen in the following image, from most fooled ones to least.

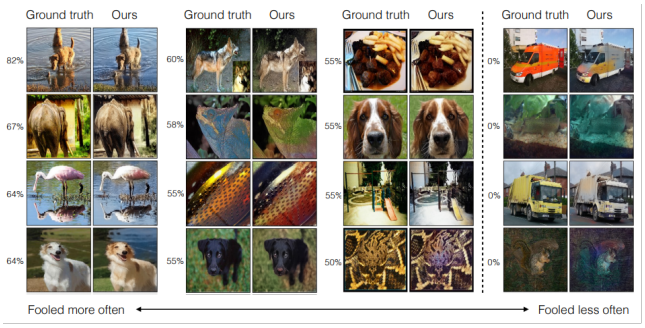

Figure 2. Some examples using Zhang's model, showing how different images where colorized

2.2 Iizuka et al (2016

Even though the architecture (and name) of this proposal is complex as it has 4 different CNNs, I would like to focus on the final "Colorization Network". Once the features are processed by the upsampling layers, the input is resized and passed through a CNN with sigmoid transfer function to get the chrominance (a,b channels). Finally, these channels are combined with the lightness (L) and the MSe is calculated with the original image.

From this model, I have learned how regression and MSE could be used to get colorization on images. However, the complexity and restricted resources I have available limit my computing capabilities, a simpler model could be performed. I have done that using a UNet architecture. It should also be noted that Iizuka also performs classification with labels that represent global image tags, to give contextual information to the model (outdoors vs indoors). This would also mean adding complexity to my model, and since they admit that regression by itself works well, I will not use it. In Figure 3 we can see some examples.

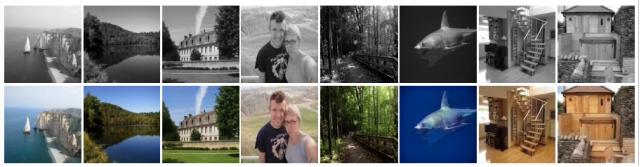

Figure 3. Some examples using Iizuka's model, that mixes regression and classification regarding contextual information

3 Methodology

In this section, we will review the different concepts that require more explanation. Some knowledge of AI related processes is required, but not very deep.

3.1 Color Spaces

One of the main aspects that image colorization requires is to define a color space to map the output pixels to. The main one used in digital artifacts is RGB, although other color spaces would seem reasonable to use, like CMYK, YCbCr, HSV or CIELab. All of them treat color, saturation, hue, lightness in different ways, but tend to maintain 3 dimensions. Therefore, we will reduce the investigation to RGB and CIELab.

3.1.1 RGB The RGB (Red, Green, Blue) color space is the most commonly used color representation in digital imaging. Each pixel

is described using three components representing the intensities of red, green, and blue channels, typically ranging from 0 to 255. While it is intuitive for display devices, the RGB space is not perceptually uniform. This means that Euclidean distances between colors does not correspond with human perceived differences. For example, humans tend to perceive red and blue more *equally distinct* as gray and white, even though the numerical difference is larger in the first case.

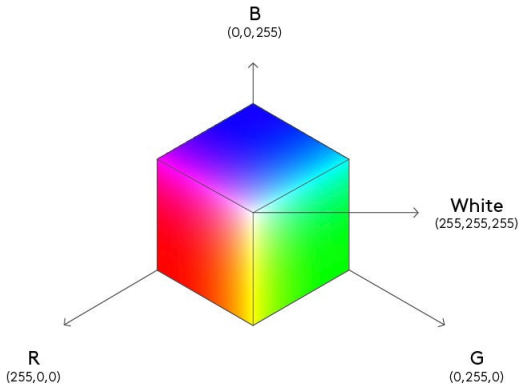

Figure 4. RGB color space

3.1.2 CIELab The CIELab (or Lab) color space is designed to be more perceptually uniform. It separates luminance (L) from chrominance (ab), which makes it highly suitable for colorization tasks. The space consists of three channels:

- L: lightness (similar to grayscale intensity), ranging from 0 (black) to 100 (white)
- a: green–red component from -128 (green) to 127 (red)
- b: blue–yellow component from -128 (blue) to 127 (yellow)

Unlike RGB, Euclidean distances in CIELab space correspond more closely to perceptual differences, making it very useful for loss calculations in colorization.

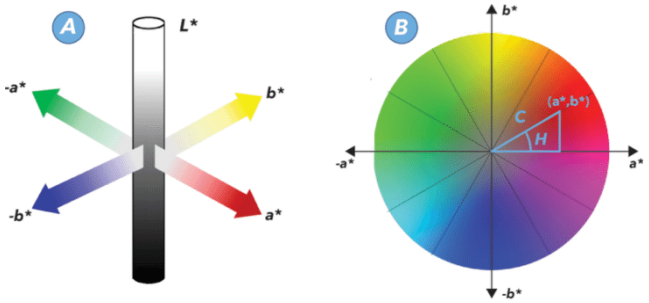

Figure 5. CIELab color space

3.1.3 Grayscale Grayscale images contain only intensity information and no color. In the Lab space, this corresponds to the L channel alone. This will be used in the models as input, with each image a size of $(1, H, W)$ following PyTorch standards.

3.2 UNet Architecture

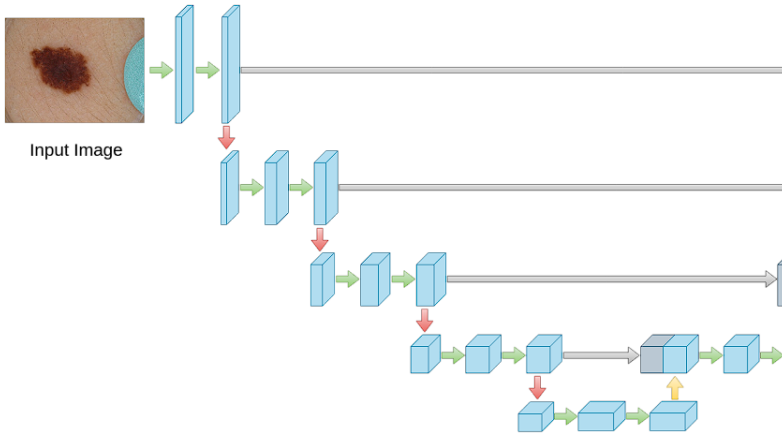

Figure 6. Figure 4. This is an example of a UNet architecture, where skipped connections have been used. It is used for mask segmentation

The U-Net architecture is a convolutional neural network that has proven effective for many image-to-image translation tasks, including colorization. It follows an encoder–decoder structure with skip connections. We can see this architecture in Figure 4.

3.2.1 CNNs Convolutional Neural Networks are the main component of U-Net architectures. A convolutional layer applies a set of filters to an input image, and moving it in the (H, W) dimensions. Each filter can be understood as a small tensor of size (C, H_k, W_k) that is moved through the input (step depends on the stride) while doing a convolution operation. This way, the model can learn features like edges or textures that can *summarize* information from the image.

The output size of a convolutional layer for H or W is given by:

$$O = \left\lfloor \frac{I + 2P - K}{S} \right\rfloor + 1 \tag{2}$$

where:

I is the input size (H, W), P is the padding,
K is the kernel size, S is the stride.

The typical architecture of CNN layers reduces spatial resolution (H, W) dimensions, while increasing the depth of feature maps (C) dimension. These are later processed by deeper layers.

3.2.2 Upsampling In the decoder part of the U-Net, upsampling layers reverse the downsampling operation by increasing the spatial resolution of the feature maps (H, W). This way, we can reconstruct the image to its original dimensions. Common techniques include:

- Transposed convolution
- Bilinear or nearest-neighbor upsampling followed by a convolution
- UnPooling (average or max)

3.2.3 Skip Connections Skip connections link encoder layers to decoder layers at the same level. They can be seen as arrows in Figure 4. They pass high-resolution feature maps directly to the decoder, preserving spatial detail and improving reconstruction quality. This is crucial in colorization, as most spatial information is passed this way There are several ways to combine encoder-decoder information, but one of the most common is by concatenating in the channel (C) axis.

3.3 Perceptual Loss

Perceptual loss is used to compare high-level features between the predicted and ground truth images, instead of comparing pixel values directly. These features are usually extracted from intermediate layers of a pretrained network like VGG19. This way, the network is able to produce visually similar features in terms of semantics and textures, improving realism.

The loss is usually computed by comparing the predicted image (output of the model) against the ground truth image. Then it is added to the main loss function of the model multplicated by a hyperparameter called λ that defined how much this loss weighs:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{L1}} + \lambda \mathcal{L}_{\text{percep}} \tag{3}$$

$\mathcal{L}_{\text{total}}$ - total loss
$\mathcal{L}_{\text{L1}}$ - L1 Loss (MAE)
$\mathcal{L}_{\text{percep}}$ - perceptual loss
λ - $\mathcal{L}_{\text{percep}}$ weight

4 Data Processing

In this section, I will describe the dataset used, discuss the exploratory data analysis I performed on some images in the dataset and the preprocessing techniques used.

4.1 COCO Dataset

The original COCO (Common Objects in Context) dataset is used for computer vision tasks like object detection, segmentation, or image captioning. It originally contains over 330,000 images, many of them containing labels for bounding boxes, sementation masks, human pose keypoints and captions

Although this dataset is very informative, I will not be using any of these labels as my task only requires a grayscale image as input and a colored images as ground truth. Alos, given the large scale of the dataset, I have used a smaller version that has around 30,000 images.I have divided that into a 90% training set and 10% validation set.

Figure 7. Some examples from the MS-COCO dataset

4.2 Exploratory Data Analysis

Even though we do not have labels to see how they are distributed, we can do so with pixel values. We will analyze 1k random images. After selection, I have transformed the images to the CIELab as it is easier to analyze with human perception.

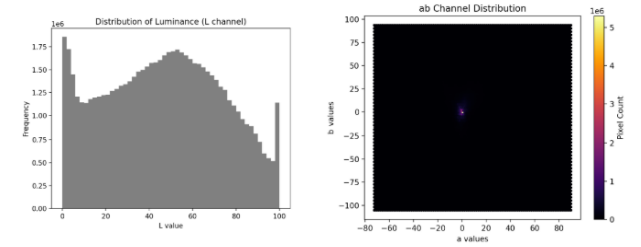

Figure 8. Some examples from the MS-COCO dataset

Although the black plots might seem quite uninformative, it actually tells us a lot of information about the *colorfulness* of each image. These are not usually very bright, which can be seen with most (a, b) valyes being located in the bright spot in the middle. However, the L channels is much more distributed with comprehensible peaks in the extremes and the middle. This is what was seen in *Zhang et al (2016)* and why he decided to perform weighted cross-entropy and add more diversity to barely seen colors.

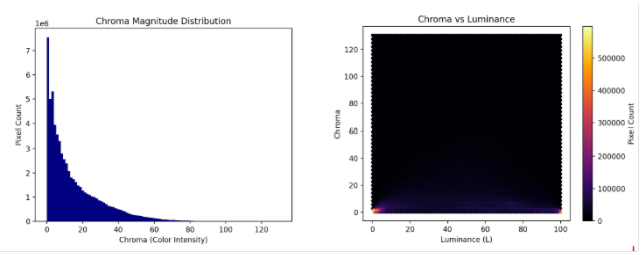

Figure 9. Some examples from the MS-COCO dataset

For this I have computed the chroma (colorfulness, saturation) of each pixel with the following formula:

$$C^* = \sqrt{a^{*2} + b^{*2}} \tag{4}$$

This confirms what I inferred from Zhang, that images are quite unsaturated (values are skewed towards the left in the first plot) and that there is not much relationship with Luminance (L). We have maximums in 0L and 100L, which makes sense, and then distributed following the grayscale plot and without getting high chroma values.

4.3 Feature Engineering

This part of the training pipeline is different for regression and classification models. However, the initial step is same: image resizing.

The COCO dataset was also chosen for its variety in image sizes, encouraging the development of a model that generalizes well across different resolutions. To standardize training, a hyperparameter `image_size` is introduced to resize inputs to a fixed dimension. Larger values of `image_size` yield higher visual quality but increase computational cost. This trade-off can be partially offset by using larger kernels in the initial convolutional layers to reduce spatial dimensions early in the network.

The next step we must differentiate between the color spaces. If we are using...:

- **RGB** then we will transform the original resized image to grayscale with dimensions $(1, H, W)$ and to RGB of dimensions $(3, H, W)$.
- **CIELab** then we define the input as the L channel with dimensions $(1, H, W)$ and the target image with channels $(2, H, W)$, as we only need channels (a,b) to define color.

The next step for both color spaces is transforming the image to tensors so it can be processed by the model. Have in mind that batch dimension have not been included, but exist as (B, C, H, W) following PyTorch standards.

Moreover, classification task requires an additional on target images step before computing the loss. We must get the bin that specific pixel with (a,b) components falls into. That is finding the closest centroid in a k-means clustering algorithm (will be explained later). This gives us the target bin for that specific pixel.

An additional data processing consideration arises from using skip connections, which double the number of channels at each decoder level due to concatenation. While it's possible to match these dimensions precisely with the corresponding encoder layer, a simpler approach is to set a target number of channels and apply interpolation—such as bilinear upsampling—to align dimensions as needed.

5 Models

In this section, I will be explaining what hyperparameters I chose for each model, how the training process and analysis on the results. We will review a UNet architecture using RGB regression and a similar model to Zhang's proposal using CIELab and quantization classification task. Both models have been designed incrementally, so I will be starting with a base model and explain the innovations implemented in posterior versions.

5.1 UNet Architecture

As mentioned, this model has been a UNet architecture following a similar structure as Figure 4. However, in my case each Conv2d block was built with a Conv2d layer, followed by Batch Normalization and an activation. We will use several of these blocks, and then perform regression on the RGB color space. That means, we need to upscale the channel dimensions from 1 to 3 while preserving H and W. I have chosen the RGB space as the values in (a.b) have shown to be very concentrated around 0 and thought the model would find a thought time learning. RGB seems to be more widely distributed.

This is the general structure. I will not go into detail with concrete hyperparameter values, only the ones I believe have driven specific outcomes.

5.1.1 Basic UNet Model For the basic model, I did some high-level testing for my initial parameters at the beginning regarding activation functions and decided that LeakyReLU worked the best in my case. Another important characteristic is that I made quite a shallow model of 5 layers and 32 filters (channels) along with an `image_size` of $(256, 256)$. The loss function used was MAE (L1). Lastly, I decided to prevent overfitting by using dropout 0.2 and a scheduler to prevent getting stagnated in local minima. One representative example of results is:

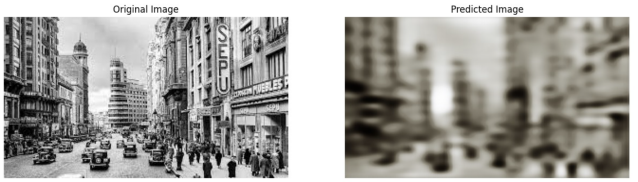

Figure 10. Attempted colorization of old Gran Via in Madrid.

As it was mentioned in the introduction, this model tends to make sepia-tinted images while losing a lot of quality. There could be several reasons behind this. First of all, regression on the RGB color space is a very difficult task as you need to gain information. Moreover, it is true that since this is my first approximation, I did not use skipped connections and the `image_size` was only (256,256). These could be the main reasons for its blurriness. Still, what the model interprets as minimizing the distance between a prediction and ground-truth is a very unsaturated image.

5.1.2 Lets dig deeper... To try and solve some of these problems, I implemented the skipped connections and set the initial `image_size` to (512, 512). Since this would make the model much heavier to compute, I also decided to insert a couple more layers with 32 channels at the beginning of the encoder (and end of decoder) that included `stride=2`. This means that the (H, W) dimensions would get smaller and less resources required.

Figure 11. (Wrong) Colorization of Real Madrid squad in the 1950s at Santiago Bernabeu

In this image at least I have some color. It can be seen that thanks to the skipped connections and `image_size` increment, quality has improved. The sepia-tones still prevail due to an averaging effect, but the most extreme grayscale values have been mapped to a color. From the lighter parts of Real Madrid t-shirts, we can see the model interprets them as white even though they should still be white. Also, the darkest part of the image have been identified as green or blue. These colors tend to appear more in darker tones compared to red. Overall, this seems as a general saturation issue, although it does not tend to appear when using LeakyReLU.

5.1.3 I am getting lost To solve this issue I saw some research that implemented perceptual loss. I took the VGG19 pretrained model, froze the gradients and computed the loss between the output of my model compared to the ground-truth. This was then added to the L1Loss multiplied by $\lambda = 0.3$

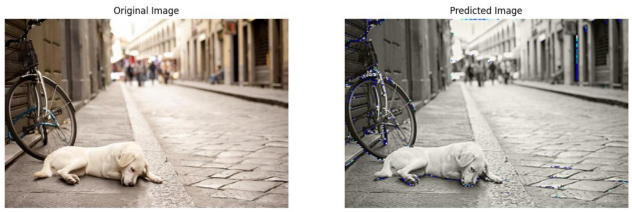

Figure 12. Gray-Colorization of of a stranded dog

I had higher expectations for this model... that went away quite quickly. Instead of the sepia-tones dominating the image, it turned grayer. The very little colors we could see are from the darker areas that have mapped out to bright green or blue. This could be due to perceptual loss prioritizes texture over colors. However, that means that even when optimizing λ, we would be moving between sepia and gray image tones.

5.2 Zhang Model

For this model I have followed the architecture offered by *Zhang et al (2016)* shown in Figure 1. In this case I use the CIELab. From there I take the L channel as grayscale and (a,b) as target. As mentioned, we now need to perform the bin quantization to get the output bins for class probabilities.

5.2.1 313 Bin Quantization To get the class weights, Zhang created 313 color bins using k-means clustering ($k = 313$) in the ab color space. This is the solution for a clearly imbalanced space. When creating the dataset, it is important to assign a bin to each (a,b) pixel. This is done by finding the nearest centroid using Euclidean distance. In this phase I encountered an important resources problem. My machine was quite slow when computing the k-MeansClustering algorithm, so I had to randomly select 1,000 pixels from each image and batches.

Afterwards, the Zhang rebalancing formula is applied, so rare bins get higher weights than it appears in the dataset. Most frequent colors will appear less. Lastly, the weights will be used for the Weighted Cross-Entropy loss function.

5.2.2 Base Model For this model version I followed Zhang proposal and their parameters. The number of bins is defined to be 313 and they use `sgd` with `momentum=0.9`. Also `image_size` is set to $(256, 256)$ and starts with a lower batch size.

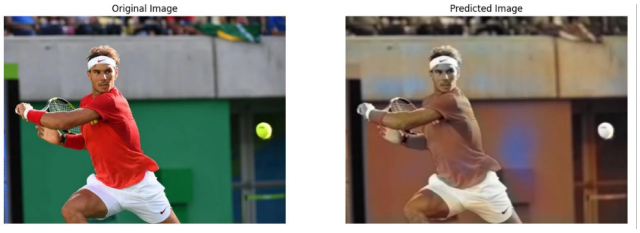

Figure 13. Attempted colorization of Rafa Nadal playing tennis

The results of the class weights is now clearly seen in the results. In this Rafa Nadal image we can see how there are orange and blues predominance. However, it is true that there are many colors that have not been identified at all, like green.

5.2.3 Make weights weigh In this case, and since I followed Zhang's proposal, I did not change many parameters in the model. Only upgraded `sgd` to `adam` and increasing the `batch_size` to stabilize training and make it faster. I also included a scheduler so learning rate gets reduced over time. The biggest change was that I increase the number of random pixels used when getting the quantized bins, as this is the main source of coloring in the image.

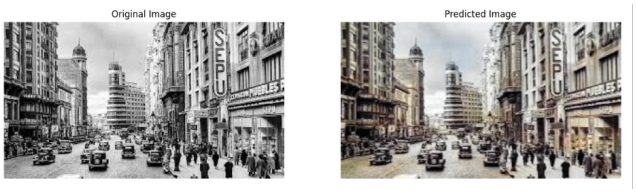

Figure 14. (Much better) Colorization of old Gran Via in Madrid

The results in this old Madrid image have been much better, even though it is still a bit unsaturated. Changes have made its effect. Training time was reduced and more colors appear in the image. However, and compared to the UNet architecture, there is a noticeable loss of quality due to the reduced `image_size`

6 Discussion

In this section I will discuss the main insights gained from the model, in terms of strengths and limitations of the qualities seen.

1. **Regression is 3x faster to train**. Even though the regression task had one more channel to predict the output to, it took around 3x less per iteration (and epoch). The main reason behind this is that when processing the data for the Zhang model, you need to transform it to RGB first, then to CIELab and lastly quantize the (a,b) channels. These extra two steps (specially image space transformation) tend to take a lot of resources. When analyzing the GPU consumption during training, it looked like a high amplitude sinusoidal function. In the peaks, the engine was processing a batch and in the troughs the images were being created and transformed.
2. **Classification achieves better results.** If we compare the images from the UNet architecture and Zhang model, we can see that we have achieved more accurate colorization. Specially, this is important because we deviated from the sepia-toned images from UNet. The rationale behind this, more than the color space and the model itself has been the weighted cross-entropy, as this diversifies more the color spectrum.
3. **Really resource-daming tasks.** One of the main problems I encountered with both models is training them in a timely manner. Computer Vision tends to take up a lot of memory, and GPU availability is a need. Even though I managed to use a free Lightning AI account to gain access to a T4 GPU, training times were still on the 10-12 minute range per epoch. This is with only with 30,000 images. For reference, Zhang trained their model in ImageNet with 1.3M images.
4. **Upscaling tasks are tough.** There are hundreds to millions combinations of (R, G, B) combinations that give a concrete greyscale value. The same happens with the CIELab color space. The model needs to learn information from other

things: shapes, contextual information through labels or narrowing the color possibilities. These techniques have been used in Zhang and Iizuka papers, proving to be very useful. However, this information is not always available, or the model is not well enough, it just tries to minimize the loss function without *seeing* the context.

6.1 Future Work

This report has analyzed two different methods that seem to have worked in other cases. It can be seen that RGB color space does not need to be used as there is a simpler one available (CIELab - only 2D color dimensions). Also, there are other novel approaches to image colorization tasks like CGANs mixed with UNet architectures. This one is used in the DeOldify model powering a MyHeritage tool.

Another widely used idea could be fine-tuning a state-of-the-art vision model and specialize it in image colorization. In this case, we could take for example VGG19, unfreeze some of the first or last layers and train over our loss function. We could also add additional CNN layers to upscale to the desired sizes.

Also, One of the main drawback has been the computer resources I had compared to other papers. Computer Vision tasks require A LOT of data, A LOT of time and A LOT of resources. And I had to find a trade-off between the three within the limitations.

7 Conclusion

In this project I have explored two different approaches to grayscale image colorization. The first method included using a UNet architecture in the RGB color space, and the second technique resulted from the findings by Zhang et al.'s in the CIELab space with quantized bins. Through incremental model development and experimentation, I have observed that while regression tends to be more efficient overall and simpler to implement, it tends to produce sepia-toned results. In contrast, classification model achieved more realistic and diverse colorization, mainly due to the use of a weighted cross-entropy loss and the simplicity of the task.

However, improving the perceputal quality with the classification came at the expense of higher resource demands. Memory usage and image preprocessing overhead when transforming into the CIELab color space slowed down the training process noticeably. Moreover, the challenges inherent to colorization tasks like the vast ambiguity between a grayscale value and its possible color space combinations. The results confirm that while direct regression can serve as a lightweight model, classification approaches offer superior visual fidelity for complex tasks like image colorization. This is more pronounce when perceptual priors and rebalancing strategies are used, since they are only possible in classification. Future work in this research topic could include semantic and contextual understanding combined with adversarial training to enhance realism and richness of generated colors.

8 Bibliography

References

[1] Lin, T.-Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). *Microsoft COCO: Common Objects in Context* [Data set]. arXiv. https://arxiv.org/abs/1405.0312

[2] UCSC-VLAA. (n.d.). *Recap-COCO-30K* [Data set]. Hugging Face. https://huggingface.co/datasets/UCSC-VLAA/Recap-COCO-30K

[3] Zhang, R., Isola, P., & Efros, A. A. (2016). Colorful image colorization. *arXiv preprint arXiv:1603.08511*.

[4] Iizuka, S., Simo-Serra, E., & Ishikawa, H. (2016). Let there be color!: Joint end-to-end learning of global and local image priors for automatic image colorization with simultaneous classification. *ACM Transactions on Graphics*, *35*(4), Article 110. https://doi.org/10.1145/2897824.2925974

[5] Khan, M. (2024, November 23). Deep learning techniques for image colorization: A step-by-step guide. *DataDrivenInvestor*. https://medium.datadriveninvestor.com/deep-learning-techniques-for-image-colorization-a-step-by-step-guide-66c5a4504877

[6] Antic, J. (2018). *DeOldify: A Deep Learning based project for colorizing and restoring old images (and video!)* [Computer software]. GitHub. https://github.com/jantic/DeOldify

[7] Zayd, M. H., Yudistira, N., & Wihandika, R. C. (2022). *Image Colorization using U-Net with Skip Connections and Fusion Layer on Landscape Images*. arXiv. https://arxiv.org/abs/2205.12867